

Evaluación final de módulo

Web API con JavaScript y JSON Server

Objetivo: implementar métodos JavaScript que consuman una Web API utilizando fetch().

1. Lanzamiento de la actividad

1. Abra el proyecto SD-12-1 AssessmentServer-Individual en VS Code.
2. Abra una terminal dentro de la carpeta del proyecto y ejecute:

```
node index.js
```

3. Verifique que el servidor indique que está listo.

El servidor estará disponible en una dirección similar a: `http://localhost:3000`. El puerto puede variar según la configuración del proyecto.

Importante: mantenga el servidor ejecutándose durante la evaluación. Antes de correr pruebas que modifiquen datos, reinícielo para restaurar la base de datos inicial.

2. Ejemplo de una solicitud con Fetch

La API Fetch permite realizar solicitudes HTTP desde JavaScript.

```
fetch("http://localhost:3000/users", {  
  method: "POST",  
  body: JSON.stringify({  
    id: 6,  
    first_name: "Alicia",  
    last_name: "Miranda",  
    email: "alicia@example.com"  
  }),  
  headers: {  
    "Content-Type": "application/json; charset=UTF-8"  
  }  
});
```

- `fetch()` realiza una solicitud HTTP.
- `POST` indica que se quiere crear un nuevo recurso.
- `JSON.stringify()` convierte un objeto JavaScript en texto JSON.
- `Content-Type: application/json` indica al servidor el formato de los datos enviados.

Nota: La URL del ejemplo es solamente una referencia. Durante la evaluación debe utilizar la URL real de su propio servidor.

3. Tareas

Una Web API permite que las aplicaciones se comuniquen con un servidor, soliciten recursos y reciban respuestas. En esta evaluación escribirá métodos JavaScript que interactuarán con una API generada mediante json-server.

Campo	Descripción	Ejemplo	Tipo
id	Identificador único	6	Número
first_name	Nombre	Alicia	Texto
last_name	Apellido	Miranda	Texto
email	Correo electrónico	alicia@example.com	Texto

Trabjará en el proyecto: SD-12-1 Assessment-Individual

- task1.js
- task2.js
- task3.js
- task4.js

No modifique index.js.

Tarea 1 — Obtener la URL del servidor

Cree y exporte el método:

```
getServerURL()
```

El método debe devolver la URL base de su servidor JSON en funcionamiento, por ejemplo:

```
http://localhost:3000
```

No incluya /users en la URL base.

Tarea 2 — Listar usuarios

Cree y exporte el método:

```
listUsers()
```

- Realizar una solicitud al recurso /users.
- Obtener la respuesta JSON.
- Imprimir en consola la lista completa de usuarios.

Tarea 3 — Agregar un usuario

Cree y exporte el método:

```
addUser(first_name, last_name, email)
```

El método debe recibir tres datos y agregar un usuario completo al servidor.

```
addUser("Alicia", "Miranda", "alicia@example.com")
```

- El id debe ser único.
- El id debe ser secuencial.
- El nuevo id debe ser mayor que el id más alto existente.

Ejemplo: si existen los ids 1, 2, 3, 4 y 5, el siguiente usuario deberá tener id 6.

Tarea 4 — Eliminar un usuario

Cree y exporte el método:

```
delUser(id)
```

El método debe recibir un número de identificación y eliminar al usuario correspondiente.

```
delUser(5)
```

4. Pruebas

Antes de ejecutar las pruebas de una tarea, reinicie el servidor.

En la terminal del servidor, deténgalo con:

```
Ctrl + C
```

Después vuelva a ejecutar:

```
node index.js
```

Desde el proyecto del cliente puede ejecutar las tareas con:

```
node index.js 1
node index.js 2
node index.js 3
node index.js 4
```

5. Métodos HTTP utilizados

Método	Uso en la evaluación	Recurso
GET	Consultar usuarios	/users
POST	Agregar un usuario	/users
DELETE	Eliminar un usuario	/users/:id

6. Puntuación

El instructor calificará la evaluación al finalizar la sesión. La evaluación verificará el funcionamiento de cada una de las cuatro tareas.